EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.

- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
  pip install Flask
- **Requests** – For HTTP communication between nodes
  pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
  - datetime
  - jsonify
  - hashlib
  - uuid4
  - urlparse
  - request

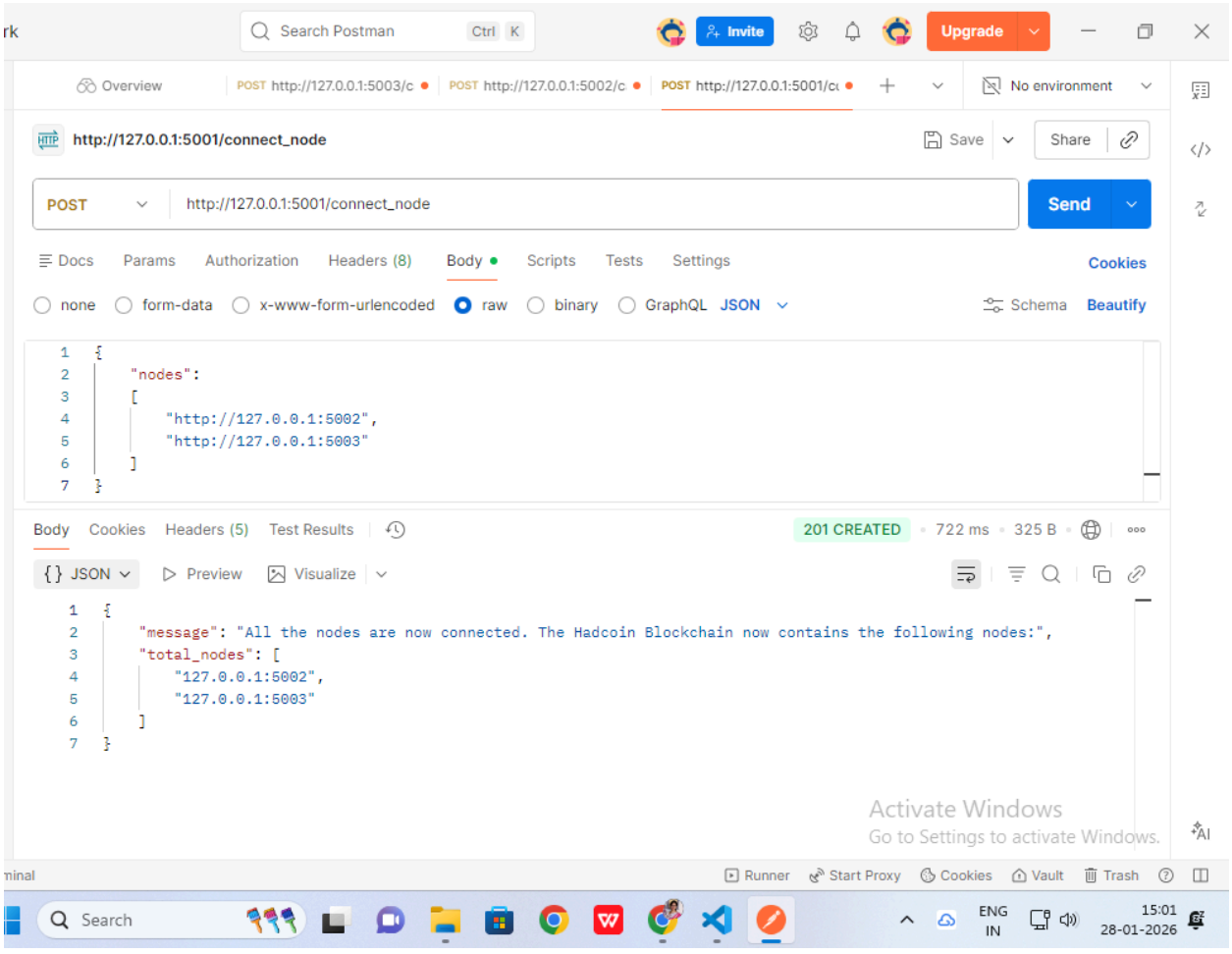
Procedure and Output:

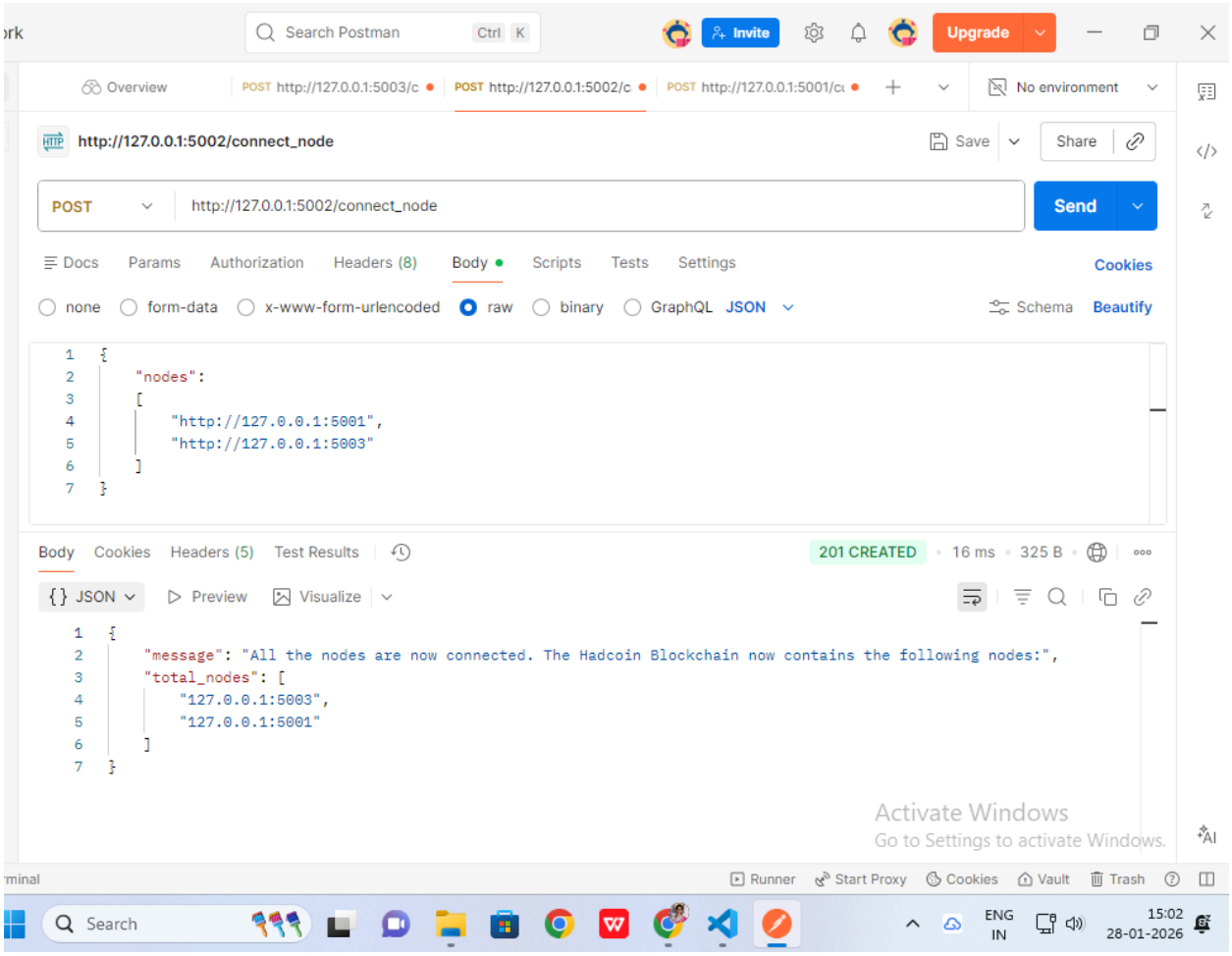
1. Download hadcoin_node_5001.py, hadcoin_node_5002.py, hadcoin_node_5003.py.
2. Install required packages.

3. Run each node in separate terminals.
4. Connect nodes using Postman – POST /connect_node.

```
{
  "nodes":
  [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5002"
  ]
}
```

5.





6. Add transactions – POST
7. Add 3 transactions in 5001

STEP 2 — Perform all transactions ONLY from one node (e.g., 5001)

POST → http://127.0.0.1:5001/add_transaction

{

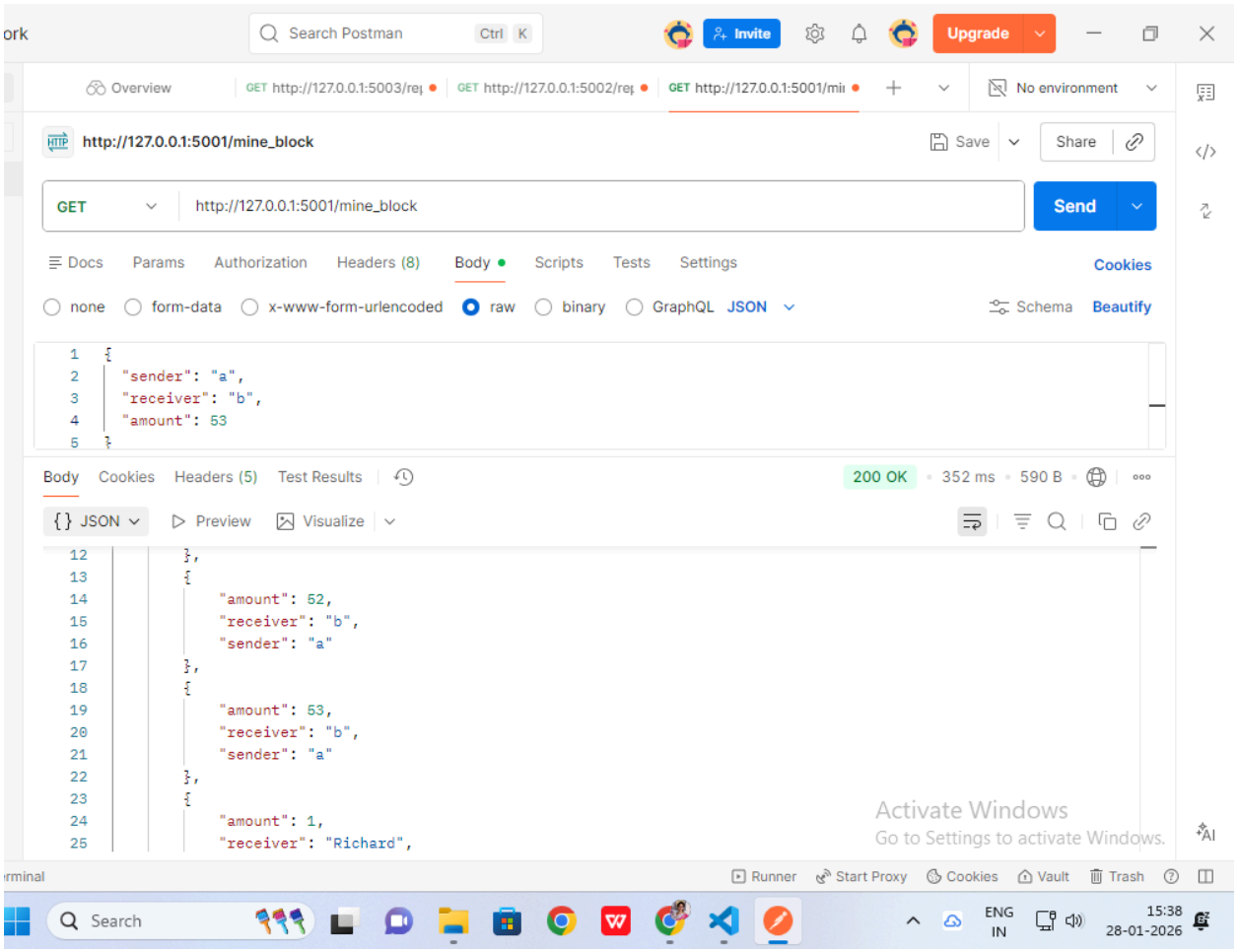
"sender": "a",

"receiver": "b",

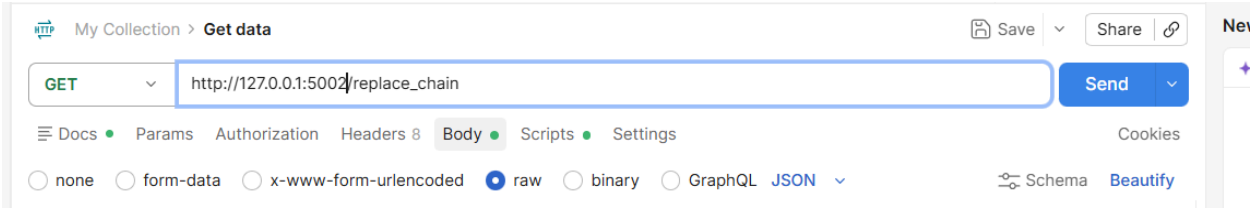
"amount": 5

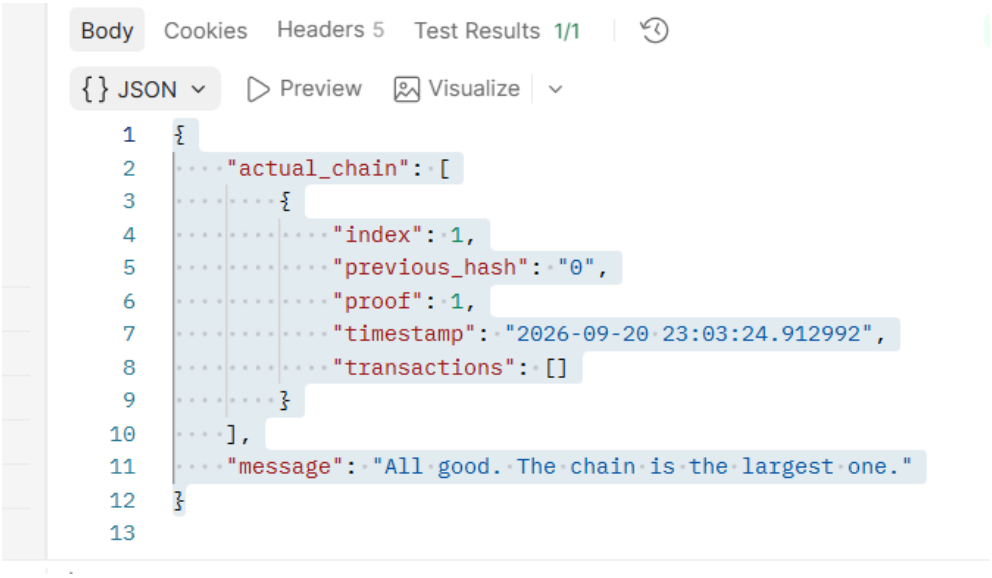
}

Then 5001 → GET /mine_block

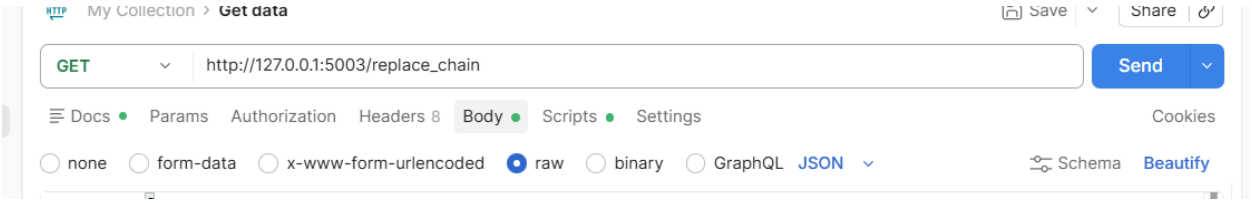


STEP 3 — Now go to 5002 and run:





STEP 4 — Repeat on 5003





Output

{

"index": 3,

"message": "Congratulations, you just mined a block!",

"previous_hash": "d501f26a58a2bfccd173a1da675769cae717df3b4536e0f2698fe8d27a45436b",

"proof": 45293,

"timestamp": "2026-01-28 15:34:33.283321",

"transactions": [

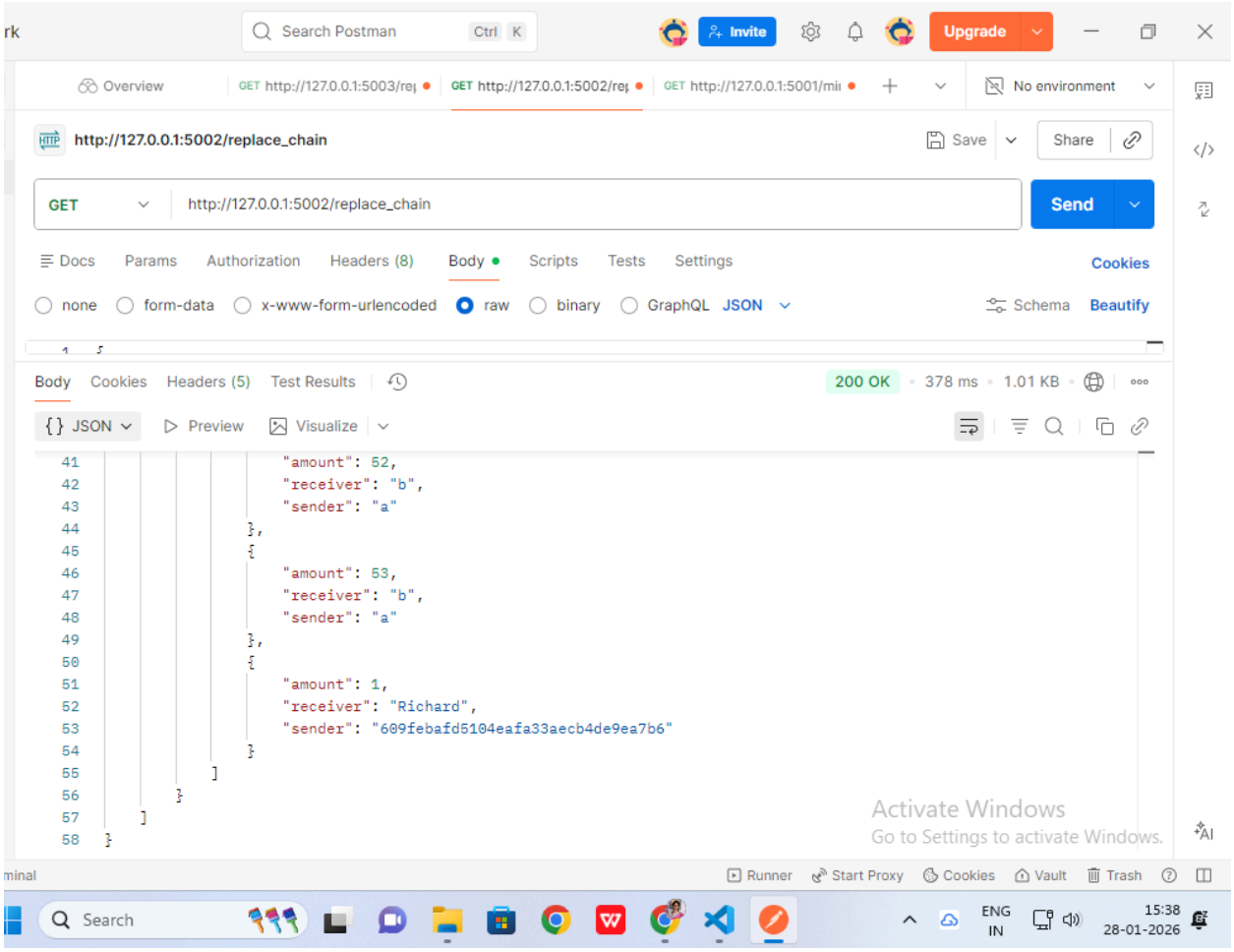
{

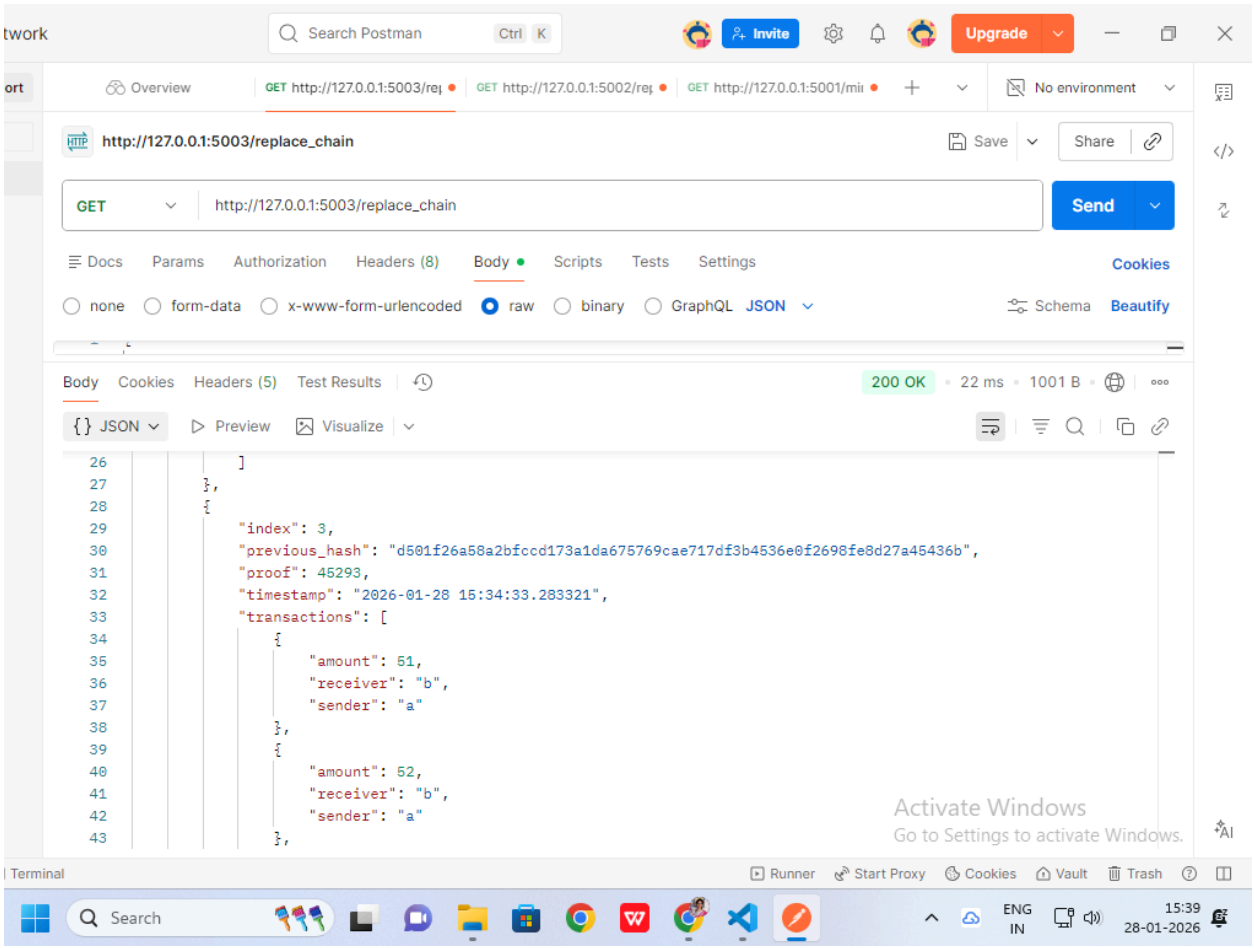
"amount": 51,

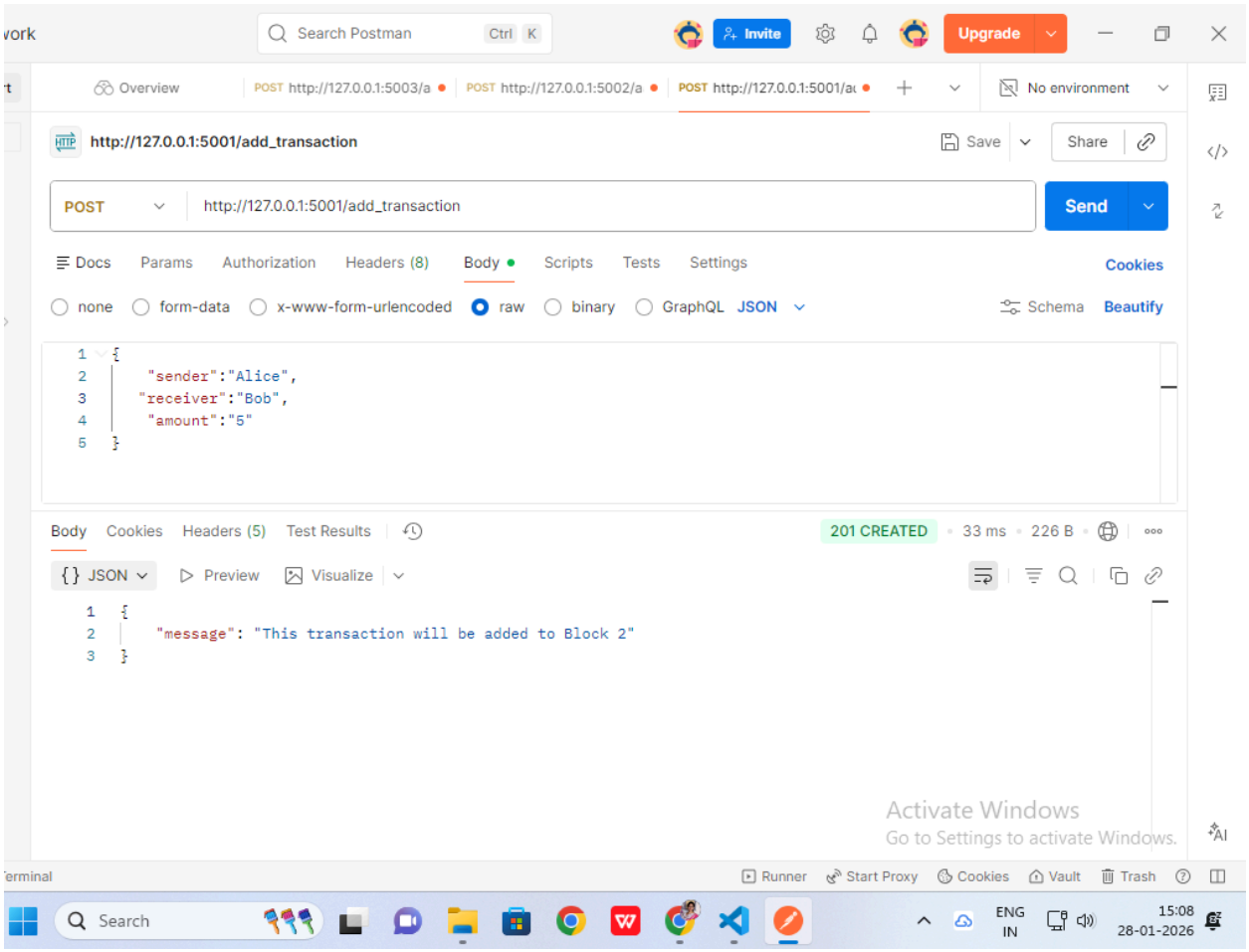
"receiver": "b",

"sender": "a"

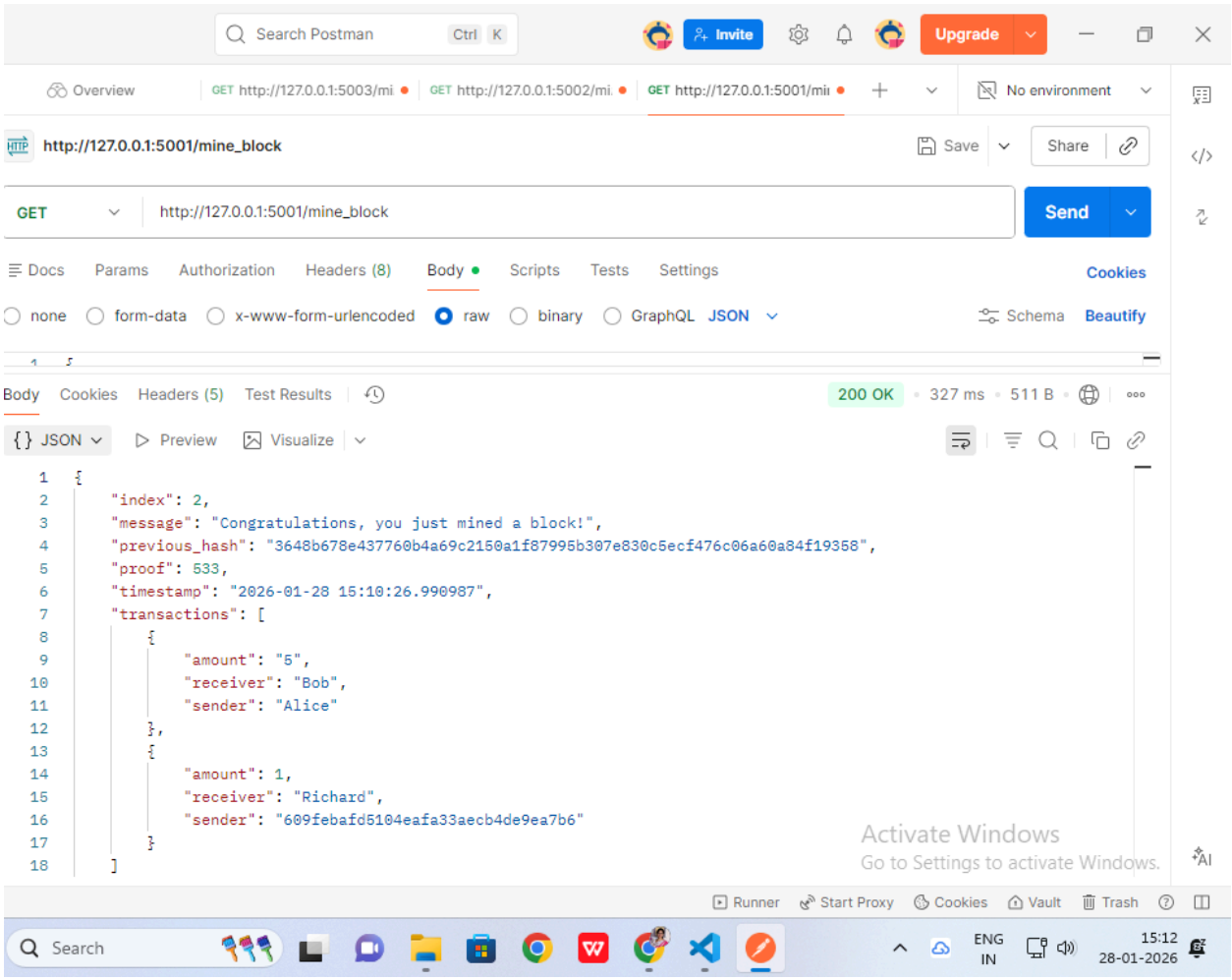
```
    },
    {
      "amount": 52,
      "receiver": "b",
      "sender": "a"
    },
    {
      "amount": 53,
      "receiver": "b",
      "sender": "a"
    },
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "609febafd5104eafa33aecb4de9ea7b6"
    }
  ]
}
```


Search Postman
Invite
Upgrade
Overview
No environment
http://127.0.0.1:5002/replace_chain
Save
Share
GET
http://127.0.0.1:5002/replace_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
378 ms
1.01 KB
JSON
Preview
Visualize
"amount": 52,
"receiver": "b",
"sender": "a"
},
{
"amount": 53,
"receiver": "b",
"sender": "a"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "609febafd5104eafa33aecb4de9ea7b6"
}
]
}
]
}
Activate Windows
Go to Settings to activate Windows.
Runner
Start Proxy
Cookies
Vault
Trash
Search
ENG
IN
15:38
28-01-2026

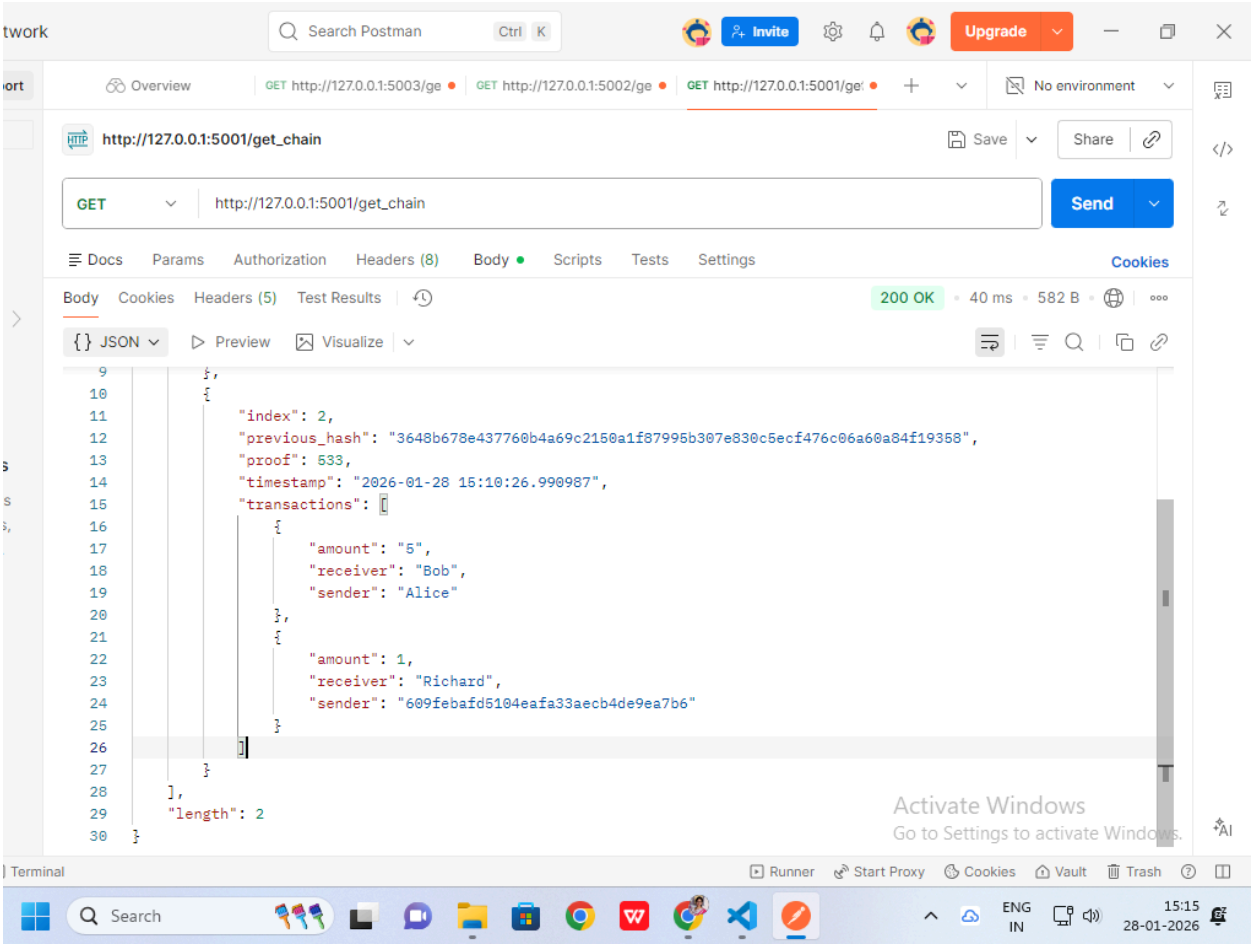

Search Postman
Invite
Upgrade
Overview
No environment
http://127.0.0.1:5003/replace_chain
Save
Share
GET
http://127.0.0.1:5003/replace_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
22 ms
1001 B
JSON
Preview
Visualize
]
},
{
"index": 3,
"previous_hash": "d501f26a58a2bfccd173a1da675769cae717df3b4536e0f2698fe8d27a45436b",
"proof": 45293,
"timestamp": "2026-01-28 15:34:33.283321",
"transactions": [
{
"amount": 51,
"receiver": "b",
"sender": "a"
},
{
"amount": 52,
"receiver": "b",
"sender": "a"
},
Activate Windows
Go to Settings to activate Windows.
Terminal
Runner
Start Proxy
Cookies
Vault
Trash
Search
ENG
IN
15:39
28-01-2026

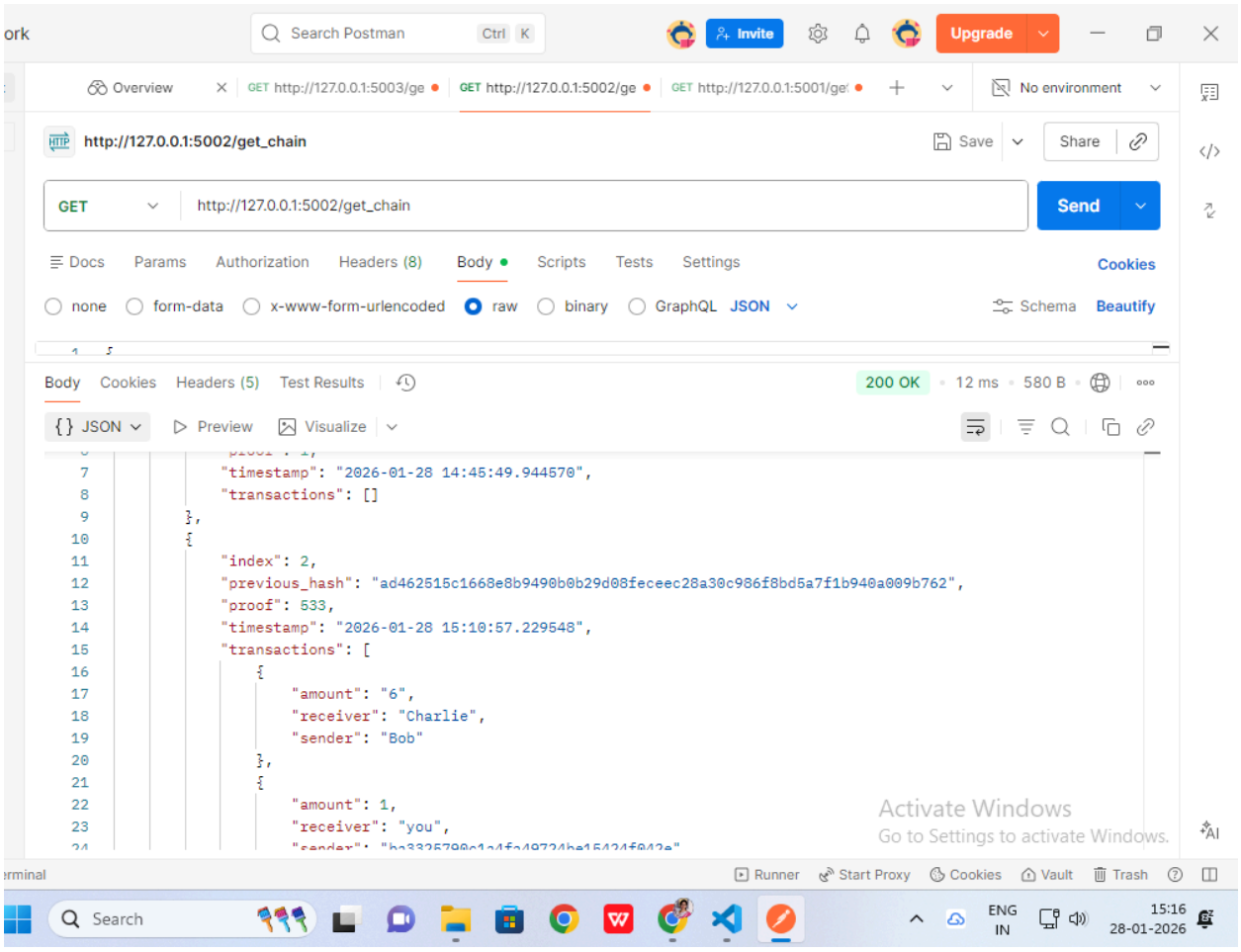


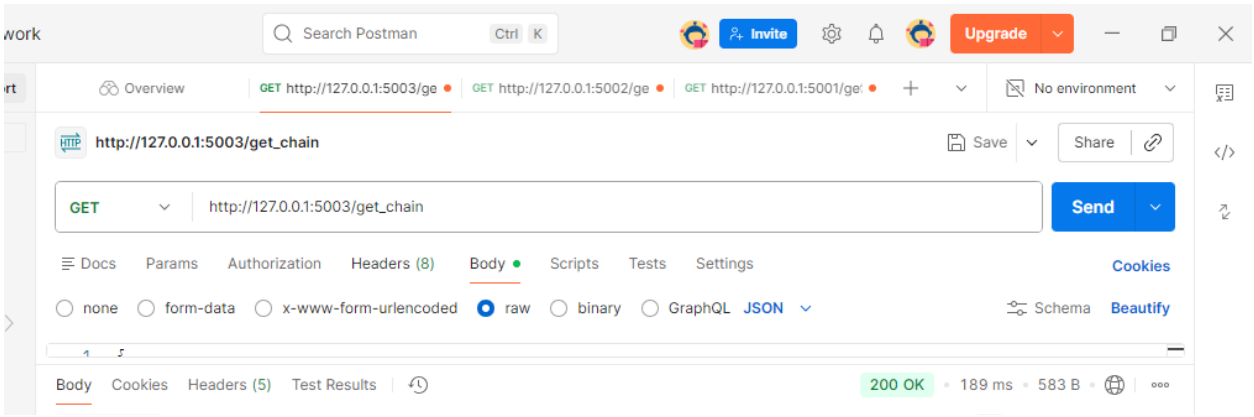
8. Mine blocks – GET /mine_block.



9. Fetch blockchain – GET /get_chain.


Search Postman
Invite
Upgrade
Overview
GET http://127.0.0.1:5003/ge
GET http://127.0.0.1:5002/ge
GET http://127.0.0.1:5001/ge
No environment
http://127.0.0.1:5001/get_chain
Save
Share
GET
http://127.0.0.1:5001/get_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
Body
Cookies
Headers (5)
Test Results
200 OK
40 ms
582 B
JSON
Preview
Visualize
{
"index": 2,
"previous_hash": "3648b678e437760b4a69c2150a1f87995b307e830c5ecf476c06a60a84f19358",
"proof": 533,
"timestamp": "2026-01-28 15:10:26.990987",
"transactions": [
{
"amount": "5",
"receiver": "Bob",
"sender": "Alice"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "609febafd5104eafa33aecb4de9ea7b6"
}
]
}
],
"length": 2
}
Activate Windows
Go to Settings to activate Windows.
Terminal
Runner
Start Proxy
Cookies
Vault
Trash
Search
ENG
IN
15:15
28-01-2026


Search Postman
Invite
Upgrade
Overview
GET http://127.0.0.1:5003/ge
GET http://127.0.0.1:5002/ge
GET http://127.0.0.1:5001/ge
No environment
http://127.0.0.1:5002/get_chain
Save
Share
GET
http://127.0.0.1:5002/get_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
12 ms
580 B
JSON
Preview
Visualize
"timestamp": "2026-01-28 14:45:49.944570",
"transactions": []
"index": 2,
"previous_hash": "ad462515c1668e8b9490b0b29d08feceec28a30c986f8bd5a7f1b940a009b762",
"proof": 533,
"timestamp": "2026-01-28 15:10:57.229548",
"transactions": [
"amount": "6",
"receiver": "Charlie",
"sender": "Bob"
"amount": 1,
"receiver": "you",
Activate Windows
Go to Settings to activate Windows.
Runner
Start Proxy
Cookies
Vault
Trash
Search
ENG
IN
15:16
28-01-2026


Search Postman
Invite
Upgrade
Overview
GET http://127.0.0.1:5003/ge
GET http://127.0.0.1:5002/ge
GET http://127.0.0.1:5001/ge
No environment
http://127.0.0.1:5003/get_chain
Save
Share
GET
http://127.0.0.1:5003/get_chain
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
Body
Cookies
Headers (5)
Test Results
200 OK
189 ms
583 B

Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.